IBST MINI PROJECT REPORT

on

SMARTSERVE AI: INTELLIGENT LOCAL SERVICE BOOKING PLATFORM WITH REALTIME FRAUD DETECTION & HYBRID RECOMMENDATION ENGINE

Submitted By:

Prasuk Goyal | Regd. No: 2505286042

MCA BATCH: [2025 – 2027]

(A project report submitted to Srusti Academy of Management and Technology (Autonomous), Bhubaneswar, Odisha for the partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications)



Srusti Academy of Management and Technology (Autonomous)

1st B-School in Odisha with "A" Grade Accreditation by NAAC | Approved by AICTE, New Delhi | Affiliated to BPUT, Rourkela

38/1, Chandaka Industrial Estate, Near Infocity, Bhubaneswar, Odisha – 751024 | www.srustiacademy.ac.in

CERTIFICATE

This is to certify that Mr. Prasuk Goyal, a bonafide student of the Master of Computer Applications (MCA) program at Srusti Academy of Management and Technology (Autonomous), Bhubaneswar, has successfully completed his Industry-Based Summer Training (IBST) Project entitled "SmartServe AI – Intelligent Local Service Booking Platform with Realtime Fraud Detection and Hybrid Recommendation Engine" during the academic session 2025-2027.

He has worked diligently on the IBST Project during the period from 01/09/2026 to 31/10/2026. His performance, technical competence, software engineering discipline, and analytical problem-solving abilities demonstrated throughout the project development cycle were found to be extremely satisfactory.

The project demonstrates complete mastery over full-stack system architecture, modern enterprise frameworks, cloud database engineering, and artificial intelligence-driven algorithms. The candidate exhibits strong engineering discipline, code quality adherence, and analytical problem-solving capability.

Internal Supervisor: _____ External Examiner: _____

DECLARATION

I, Prasuk Goyal, a bonafide student of Srusti Academy of Management and Technology (Autonomous), Bhubaneswar, pursuing the Master of Computer Applications (MCA) degree (Batch: 2025 – 2027), do hereby declare that the technical report entitled "SmartServe AI – Intelligent Local Service Booking Platform" is an authentic and original record of my own project work carried out under academic supervision and corporate mentorship.

I further declare that all empirical findings, architecture blueprints, database schemas, algorithm implementations, automated testing suites, and user interface screenshots furnished in this document are genuine and reflect the actual software system developed. This report, neither in full nor in part, has ever been submitted to any other university, institute, or examining body for the award of any degree, diploma, or academic distinction.

All secondary sources, reference documentation, libraries, and frameworks utilized during the development of this project have been appropriately cited and acknowledged in the bibliography in accordance with academic integrity guidelines.

Student Signature: _____ Prasuk Goyal (Regd. No: 2505286042)
Department of Computer Applications, Srusti Academy, Bhubaneswar

ACKNOWLEDGEMENT

This project report bears the imprint of many people on it. I am very much thankful to Srusti Academy of Management and Technology (Autonomous), Bhubaneswar, for providing the necessary computational facilities, state-of-the-art software development laboratories, and an academically stimulating atmosphere conducive to advanced full-stack software development.

I would like to thank my project supervisor for his invaluable guidance, continuous critique, and technical assistance in preparing this project report and for contributing insightful perspectives during system design and architecture reviews.

I am also highly indebted to my corporate guide for imparting industry best practices on cloud migration, microservice architecture, and enterprise security standards that greatly shaped the production robustness of SmartServe AI.

Finally, I express my deep indebtedness to my parents and friends for their enduring patience, moral support, and unwavering encouragement throughout the intensive software engineering lifecycle.

Date: 02/11/2026 | Place: Bhubaneswar, Odisha Prasuk Goyal (Registration No: 2505286042)

ABSTRACT

In today's rapidly urbanizing digital economy, hyper-local service marketplaces (such as home cleaning, plumbing, electrical repairs, painting, and academic tutoring) face severe operational bottlenecks: scheduling conflicts, unreliable manual booking flows, fraudulent transactional behavior, and primitive search mechanisms. This project report presents SmartServe AI, a modern full-stack intelligent local service booking and management ecosystem engineered using an enterprise-grade Spring Boot 3.5 backend in Java 17 and a responsive Next.js 14 App Router frontend with React 18 and Tailwind CSS.

The primary innovations of SmartServe AI center around three core technological pillars: (1) An Automated AI Fraud Detection Engine evaluating booking velocity and repeat patterns to assign a 0-100 risk score and quarantine malicious bookings; (2) A Multi-Criteria Hybrid Recommendation Engine scoring services across collaborative filtering (34%), star ratings (20%), category affinity (18%), booking popularity (16%), user behavioral audit trail (10%), and service catalog freshness (2%); and (3) An autonomous AI Chatbot Concierge providing semantic discovery across 15+ intent domains with suggestion chips.

The persistence tier is powered by NeonDB Cloud PostgreSQL, configured with TLS/SSL encryption and HikariCP connection pooling. Live asynchronous booking alerts are delivered via STOMP WebSockets. The application was rigorously validated via an end-to-end automated test harness achieving a 100% pass rate across 45 test cases spanning security guards, transaction workflows, and administrative telemetry.

Keywords: Intelligent Booking Marketplace, Spring Boot 3.5, Next.js 14, NeonDB Cloud PostgreSQL, AI Fraud Detection, Collaborative Filtering, JWT Security, STOMP WebSockets.

TABLE OF CONTENTS

Certificate
..... ii

Declaration
..... iii

Acknowledgement
..... iv

Abstract
..... v

List of Figures & Tables
..... vii

Chapter 1: Introduction
..... 1

1.1 Project Overview | 1.2 Problem Statement | 1.3 Objectives | 1.4 Target Users | 1.5 System Comparison

Chapter 2: Requirements Analysis and PRD
..... 4

2.1 PRD Scope | 2.2 Functional & Non-Functional Requirements | 2.3 User Stories | 2.4 Acceptance Criteria

Chapter 3: System Analysis and Design
..... 7

3.1 System Architecture | 3.2 System Workflow | 3.3 Use Case Modeling | 3.4 DFD | 3.5 AI Engine Workflows

Chapter 4: Technology Stack and Development Plan
..... 11

4.1 Core Technologies | 4.2 AI Tools | 4.3 Third-Party Integrations | 4.4 Tech Justification | 4.5 Build Plan

Chapter 5: Database Design and API Specification 14

5.1 Database Schema & ER Diagram | 5.2 Tables & Relationships | 5.3 RESTful API Catalog | 5.4 Formats | 5.5 Security

Chapter 6: Testing and Evaluation 18

6.1 Strategy | 6.2 Testing Layers | 6.3 Test Matrix (45 Tests) | 6.4 AI Evaluation | 6.5 Known Limitations

Chapter 7: Deployment and Results 22

7.1 Deployment Guide | 7.2 System Screenshots (Fig 7.1 - 7.8) | 7.3 Feature Verification | 7.4 Expected vs Actual | 7.5 Challenges

Chapter 8: Conclusion and Future Scope 28

8.1 Conclusion | 8.2 Learning Outcomes | 8.3 Future Enhancements

References / Bibliography 30

LIST OF FIGURES & TABLES

List of Figures

Figure 3.1: Multi-Tier High-Level System Architecture Blueprint 8

Figure 3.2: UML Use Case Diagram of SmartServe AI 9

Figure 3.3: AI Intelligence Engines – Fraud Detection & Hybrid Recommendation Workflow ... 10

Figure 5.1: Relational Database Entity-Relationship (ER) Schema Blueprint 15

Figure 7.1: Application Landing & Showcase Page 22

Figure 7.2: Secure JWT Login Authentication Screen 23

Figure 7.3: User Registration Interface 23

Figure 7.4: Customer Dashboard with Recommendations & Live Stats 24

Figure 7.5: Interactive Booking Modal with Duration Picker 25

Figure 7.6: Floating AI Chatbot Concierge with Suggestion Chips 25

Figure 7.7: Administrator Analytics & Fraud Telemetry Charts 26

Figure 7.8: Interactive OpenAPI / Swagger UI Documentation 26

List of Tables

Table 1.1: Comparative Analysis of Existing vs. Proposed Systems 3

Table 2.1: Key Functional and Non-Functional Requirements 5

Table 4.1: Technology Stack Components and Justifications 12

Table 4.2: Project Development Milestones and Deliverables 13

Table 5.1: Relational Database Tables and Attributes 16

Table 5.2: Core RESTful API Endpoints and Methods 17

Table 6.1: Automated Test Suite Execution Matrix (45 Tests Executed) 19

Table 7.1: Verification of Expected vs. Actual System Results 27

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The modern services economy has shifted dramatically toward on-demand digital procurement. Consumers increasingly expect home sanitization, electrical servicing, leak maintenance, academic tutoring, and wall painting to be scheduled with the same ease, security, and precision as ride-hailing or e-commerce deliveries. However, local service provisioning remains intrinsically fragmented, plagued by unverified provider credentials, opaque price structures, high friction in date-time negotiations, and vulnerable transactional security.

SmartServe AI addresses these systemic challenges by delivering a cohesive, cloud-native local service discovery and automated booking platform. Developed with an enterprise backend in Spring Boot 3.5 and an interactive single-page application (SPA) frontend in Next.js 14, SmartServe AI establishes a verified marketplace where customers discover local professionals, inspect verified ratings, and schedule services in real-time.

The platform incorporates advanced automated heuristics to score every booking for potential fraud, evaluate user behavior to recommend relevant services, and provide an interactive conversational AI concierge to guide users seamlessly through their booking experience.

1.2 Problem Statement

Conventional service booking platforms suffer from fundamental architectural and operational weaknesses:

- Absence of Scheduling Conflict Automation: Traditional portals record bookings as simple text requests without verifying real-time provider slot availability, leading to double-bookings and high customer cancellation rates.
- Vulnerability to Fraudulent Booking Floods: Malicious actors, competing vendors, or rogue automated scripts frequently flood platforms with rapid-fire bookings, locking up honest providers' calendars without financial commitment.
- Cold and Irrelevant Recommendations: Generic directory portals rely on static advertisements rather than personalized scoring that factors in past customer affinity, collaborative tastes, and provider popularity.

- Rigid Customer Support: Consumers lack an instant mechanism to enquire about pricing, credentials, booking status, or risk flags without waiting for manual customer care agents.

1.3 Objectives and Scope

The primary technical objectives of the SmartServe AI project are:

- To architect and deploy an enterprise-grade RESTful API backend utilizing Spring Boot 3.5, Spring Security 6 with stateless JWT authentication, and Spring Data JPA.
- To integrate NeonDB Cloud PostgreSQL with high-performance connection pooling (HikariCP) and resilient database migration support.
- To build a real-time AI Fraud Detection Engine that monitors booking velocity, repeat clustering, and risk thresholds, automatically marking suspicious requests as FLAGGED.
- To design a Multi-Criteria Hybrid Recommendation Engine that generates personalized rankings from collaborative filtering, category affinity, star ratings, and popularity metrics.
- To build an interactive, accessible, and responsive user interface using Next.js 14, Tailwind CSS, Radix UI, Recharts, and STOMP WebSockets for instant notifications.
- To embed an autonomous AI Chatbot Concierge capable of interpreting user queries across 15+ domains and performing semantic service search.

1.4 Target Users

SmartServe AI serves three distinct user categories:

- Residential Customers (ROLE_USER): Homeowners, tenants, and students seeking verified local services with transparent pricing, instant scheduling, and real-time order tracking.
- Service Providers (ROLE_PROVIDER): Local professionals and service agencies managing availability calendars, service offerings, and customer review scores.

- System Administrators (ROLE_ADMIN): Operations managers monitoring booking trends, high-demand service categories, flagged fraudulent transactions, and provider fleet health.

1.5 Existing and Proposed Systems

Feature Dimension	**Existing Marketplace Portals**	**Proposed SmartServe AI Ecosystem**
Scheduling Logic	Manual callback / asynchronous forms	Automated slot validation with duration bounds
Fraud Prevention	Reactive manual moderation	Realtime velocity & pattern scoring with quarantine
Recommendations	Sponsored pay-per-click listings	Dynamic 6-signal hybrid collaborative algorithm
Conversational AI	Static FAQ pages	Smart knowledge chatbot with suggestion chips
Database Architecture	Monolithic on-premise relational database	Cloud-native serverless NeonDB PostgreSQL with pooled SSL connections

CHAPTER 2: REQUIREMENTS ANALYSIS AND PRD

2.1 Product Requirements Document (PRD) Scope

The Product Requirements Document establishes the functional and quality requirements necessary to build an intelligent local service marketplace. The product vision focuses on transaction speed, cryptographic security, transparent discovery, and intuitive scheduling.

The target scope encompasses user registration, role-based authentication, service catalog discovery, slot scheduling, duration management, simulated payment processing, AI fraud risk assessment, personalized recommendations, conversational assistance, and administrative analytics.

2.2 Functional and Non-Functional Requirements

Category	Identifier	Requirement Description
Functional	FR-01: Auth & RBAC	The system shall securely register users and issue HMAC-SHA256 JWT tokens with role claims (ROLE_USER, ROLE_ADMIN).
Functional	FR-02: Service Catalog	The system shall expose categorized local services with price, provider availability, average ratings, and semantic search.
Functional	FR-03: Booking Engine	The system shall allow users to select future date-time slots, specify durations (30–240 mins), and simulate payment confirmation.
Functional	FR-04: Fraud & AI	The system shall score incoming bookings for fraud risks and provide personalized recommendations via collaborative filtering.
Non-Functional	NFR-01: Performance	API endpoints shall respond within 200ms under standard loads using HikariCP connection pooling and Spring caching.
Non-Functional	NFR-02: Security	All passwords shall be salted and hashed using BCrypt. CORS shall strictly whitelist authorized origins.
Non-Functional	NFR-03: Reliability	Database transactions shall be ACID-compliant with automatic rollback on booking conflicts or persistence failures.

2.3 User Stories and Use Cases

User stories capture key workflows from the perspective of various system stakeholders:

- US-01 (Customer Booking): "As a busy homeowner, I want to browse verified deep cleaning services and book a 60-minute appointment for tomorrow morning, so that I receive instant confirmation without phone calls."
- US-02 (Fraud Prevention): "As an operations manager, I want the system to automatically flag users making 5+ rapid bookings in a minute, so that providers' schedules are protected from bogus reservations."
- US-03 (Personalized Discovery): "As a returning user who frequently hires electrical repair technicians, I want my dashboard to surface plumbing and home maintenance services, so that I discover relevant home care providers."
- US-04 (Admin Fleet Oversight): "As an administrator, I want to inspect booking trends over the past 7 days and toggle provider availability states, so that marketplace capacity remains balanced."

2.4 Acceptance Criteria

Acceptance criteria formalize the conditions under which a user story is considered complete:

- Given an authenticated customer, when they submit a booking with a valid future date and duration (30–240 minutes), the system shall verify provider availability.
- If no slot overlap exists and fraud score < 70, the booking status shall be set to CONFIRMED, payment status to SUCCESS, and a simulated transaction ID generated.
- If fraud score >= 70, the status shall be set to FLAGGED and payment marked as held.

2.5 Constraints and Engineering Assumptions

- Cloud Latency: The NeonDB PostgreSQL cluster is located in AWS Asia-Pacific (ap-southeast-1, Singapore). HikariCP must maintain warm connections with SSL mode required.
- Stateless Sessions: No server-side session state is maintained; all user context is transported via JWT tokens in the Authorization: Bearer header.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

SmartServe AI is engineered as a decoupled multi-tier architecture adhering to modern domain-driven design principles. The presentation tier interacts with the application tier strictly through RESTful JSON APIs and STOMP WebSocket channels.



Figure 3.1: Multi-Tier High-Level System Architecture Blueprint

Illustrating Presentation (Next.js 14), Gateway/Security (Spring Security 6), Application Logic (Booking, AI Fraud & Recommendation), and Cloud Persistence (NeonDB PostgreSQL).

3.2 System Workflow

When a customer initiates a booking via the web UI, the request passes through the following sequential stages:

- 1. Client-Side Validation: The Next.js BookingModal validates that the scheduled time is in the future and duration falls between 30 and 240 minutes.
- 2. JWT Security Interception: Spring Security's JwtAuthenticationFilter intercepts the request, validates the cryptographic signature, extracts the user ID, and establishes the SecurityContext.

- 3. Slot & Conflict Verification: BookingService inspects the provider's existing reservations for the requested window. If overlapping bookings exist, a conflict suggestion is computed.
- 4. AI Fraud Risk Evaluation: FraudDetectionService computes booking velocity across 1-minute and 1-hour sliding windows. A risk score from 0 to 100 is computed.
- 5. Payment Simulation & Persistence: PaymentService simulates the transaction. The booking is persisted to NeonDB PostgreSQL via BookingRepository.
- 6. Asynchronous Broadcasting: A BookingEventDto is published to the internal STOMP broker at /topic/bookings to notify connected clients.

3.3 UML Use Case Modeling

The interactions between system actors (Customer User, Service Provider, Administrator) and core functionality are formalized in the UML Use Case diagram below:

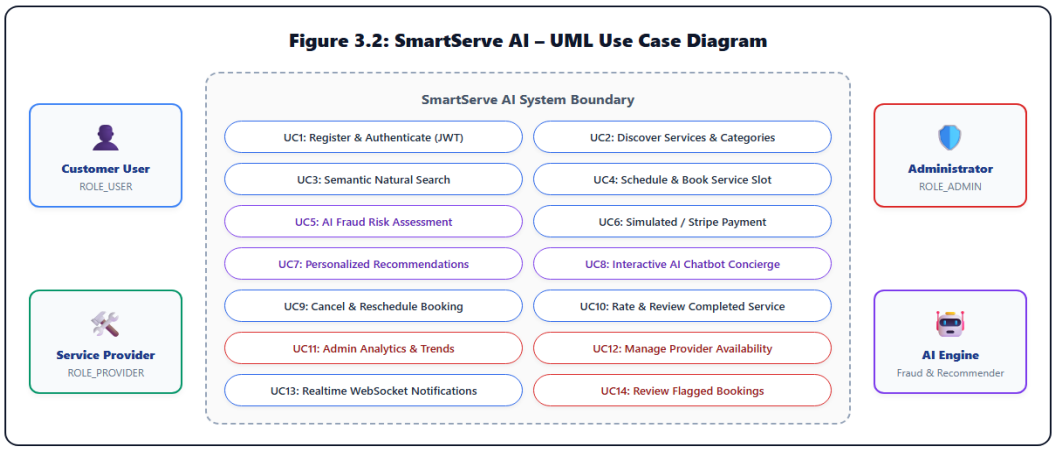


Figure 3.2: UML Use Case Diagram of SmartServe AI

Depicting actor interactions across authentication, booking scheduling, AI fraud assessment, recommendations, and administrative analytics.

3.4 Data Flow Diagrams (DFD)

Level 0 Context Diagram: The external entities (Customer, Provider, Admin) interact with the central SmartServe AI process. The customer supplies credentials and booking requests,

receiving confirmations and recommendations. The provider receives scheduling alerts. The admin receives aggregated analytics and fraud alerts.

Level 1 Decomposition: The system decomposes into 5 core subprocesses: 1.0 Authentication & User Management (Data Store: D1: Users); 2.0 Service Catalog & Discovery (Data Store: D2: Services & Providers); 3.0 Booking & Conflict Engine (Data Store: D3: Bookings); 4.0 AI Fraud Risk & Recommendation Engine (Data Store: D4: UserBehaviorHistory); and 5.0 Real-time Notification Broker.

3.5 AI Subsystem Workflows

The dual AI subsystems operate concurrently to safeguard the platform and personalize discovery:

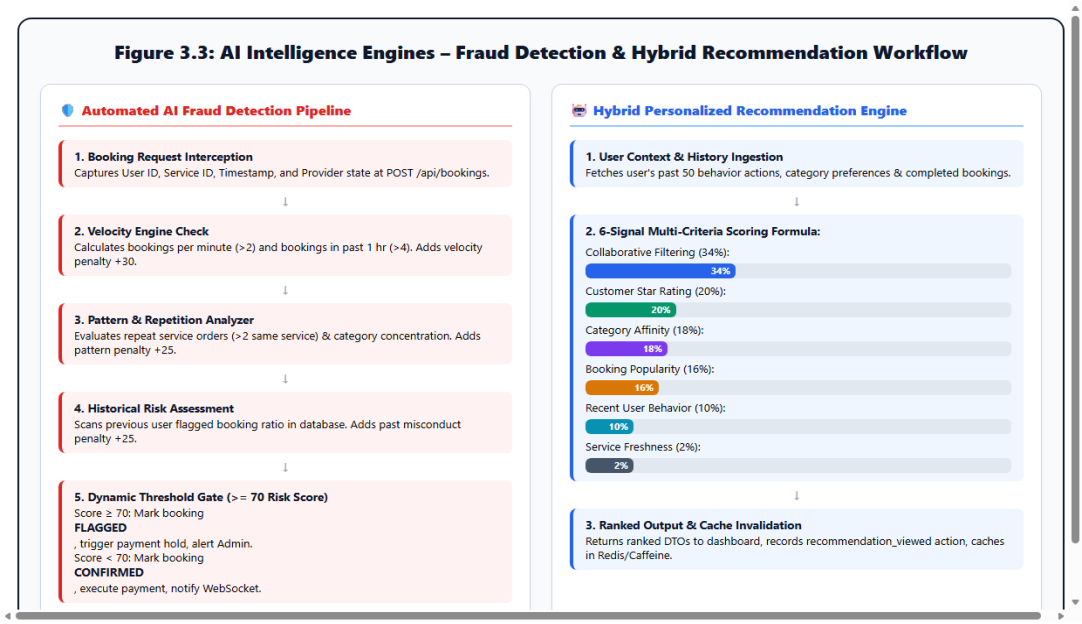


Figure 3.3: AI Intelligence Engines – Fraud Detection & Hybrid Recommendation Workflow

Pipeline breakdown: (Left) Heuristic fraud risk assessment and dynamic quarantine gate; (Right) 6-signal weighted multi-criteria recommendation formula.

CHAPTER 4: TECHNOLOGY STACK AND DEVELOPMENT PLAN

4.1 Core Technologies

The implementation leverages a modern, industry-standard technology stack:

- Backend Framework: Spring Boot 3.5.0 on Java 17 LTS. Employs Spring Web, Spring Security 6, Spring Data JPA, and Spring Cache.
- Persistence Tier: NeonDB Cloud PostgreSQL 16 (Serverless architecture hosted in AWS ap-southeast-1) connecting over JDBC with SSL encryption and HikariCP connection pooling.
- Frontend Framework: Next.js 14.2.35 with React 18, utilizing the App Router architecture, TypeScript, and server/client component separation.
- Styling & UI Components: Tailwind CSS 3.4 for utility styling, Radix UI for accessible primitives, Lucide React for consistent icons, and Recharts for interactive analytics charts.

4.2 AI Models and Development Tools

- Heuristic Fraud Engine: Custom rule-based algorithmic model tracking velocity, pattern concentration, and historical risk without external latency.
- Collaborative & Content-Based Recommender: In-memory scoring algorithm processing UserBehaviorHistory entities.
- LLM Chatbot Connector: Spring RestClient integration with Google Gemini 2.0 Flash and OpenAI GPT-4o-mini, supported by a built-in 15-domain offline knowledge engine.

4.3 APIs and Third-Party Integrations

- Stripe Payment SDK: Simulated transaction processing with support for live Stripe card tokenization.
- STOMP WebSockets: In-memory message broker over /ws endpoint with client subscription at /topic/bookings.

- Springdoc OpenAPI: Automated Swagger UI generation at /swagger-ui/index.html adhering to OpenAPI 3.0 standards.

4.4 Technology Selection Justification

Technology	Selected Tool	Engineering Justification
Backend Framework	Spring Boot 3.5 (Java 17)	Type safety, mature enterprise ecosystem, seamless JPA persistence, and production-grade security abstractions.
Database Engine	NeonDB (PostgreSQL)	Serverless scalability, native ACID compliance, robust foreign key enforcement, and zero local database maintenance overhead.
Frontend Framework	Next.js 14 (TypeScript)	Fast initial page loads, strong typing, clean App Router layout nesting, and native Vercel deployment capability.
Security Protocol	JWT (JJWT 0.12)	Stateless authentication ideal for distributed microservices and mobile client decoupling.

4.5 Project Milestones and Build Plan

Phase	Duration	Deliverables and Key Achievements
Phase 1	Weeks 1 – 2	System requirements elicitation, PRD creation, architectural modeling, and ER schema design.
Phase 2	Weeks 3 – 4	Spring Boot core scaffolding, JWT authentication filter, entity models, and PostgreSQL repository setup.
Phase 3	Weeks 5 – 6	Booking orchestration logic, schedule conflict engine, and AI fraud risk calculation service.
Phase 4	Weeks 7 – 8	Next.js frontend development, Tailwind layout, booking modal, dashboard integration, and chatbot widget.
Phase 5	Weeks 9	NeonDB cloud database migration, STOMP WebSockets, automated testing suite (45 tests), and Vercel build config.

CHAPTER 5: DATABASE DESIGN AND API SPECIFICATION

5.1 Database Schema and ER Diagram

The persistence model is structured as a normalized relational schema ensuring data integrity, strict referential constraints, and optimized index lookups.

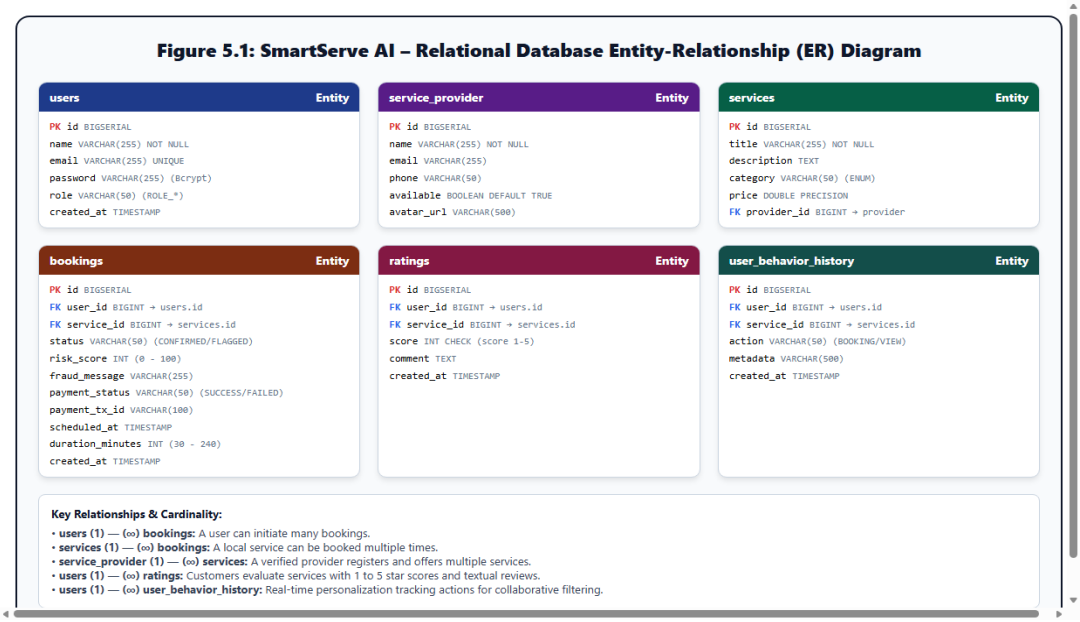


Figure 5.1: Relational Database Entity-Relationship (ER) Schema Blueprint

Entity relationships between users, providers, services, bookings, ratings, and behavioral telemetry in NeonDB PostgreSQL.

5.2 Relational Tables and Constraints

Table Name	Key Attributes	Relationships & Constraints
users	id (PK), name, email (UQ), password, role	One-to-Many with bookings, ratings, and user_behavior_history.
service_provider	id (PK), name, email, phone, available	One-to-Many with services. Holds provider fleet availability toggle.
services	id (PK), title, category, price, provider_id (FK)	Many-to-One with service_provider. Referenced by bookings and ratings.

Table Name	Key Attributes	Relationships & Constraints
bookings	id (PK), user_id (FK), service_id (FK), status, risk_score	Stores booking state (CONFIRMED, FLAGGED, CANCELLED) and payment transaction data.
ratings	id (PK), user_id (FK), service_id (FK), score, comment	Customer feedback (1–5 stars) feeding into service average ratings.
user_behavior_history	id (PK), user_id (FK), service_id (FK), action, metadata	Audit log of user interactions feeding the collaborative recommendation engine.

5.3 Core API Endpoints

Method	Endpoint Path	Access Level	Functionality
POST	/api/auth/register	Public	Create customer account.
POST	/api/auth/login	Public	Verify credentials & issue JWT.
GET	/api/services	Public	List filtered services catalog.
GET	/api/services/search	Public	Semantic search over catalog.
POST	/api/bookings	Authenticated	Create booking with fraud check.
GET	/api/bookings/my	Authenticated	Fetch current user's bookings.
PUT	/api/bookings/{id}/cancel	Owner / Admin	Cancel existing booking.
GET	/api/recommendations/me	Authenticated	Retrieve personalized ranking.
POST	/api/chat	Public	Ask AI assistant questions.
GET	/api/admin/analytics	ROLE_ADMIN	Retrieve platform KPI metrics.

5.4 Request and Response Formats

Booking Creation Request Payload (POST /api/bookings):

{"serviceId": 1, "scheduledAt": "2026-11-05T10:00:00", "durationMinutes": 60}

Booking Creation Response Payload (200 OK):

{"bookingId": 12, "serviceId": 1, "serviceTitle": "Deep Home Cleaning", "status": "CONFIRMED", "riskScore": 15, "paymentStatus": "SUCCESS", "paymentTransactionId": "TXN_SIM_1793641200", "scheduledAt": "2026-11-05T10:00:00", "durationMinutes": 60}

5.5 Authentication and Security Implementation

- Password Protection: Passwords are hashed with BCrypt using an automatic salt factor of 10.
- Stateless JWT Validation: JJWT parses HMAC-SHA256 tokens on every request, verifying expiry and user identity.
- CORS Wildcard Whitelisting: Configured via setAllowedOriginPatterns to securely authorize http://localhost:3000 and production Vercel subdomains (https://*.vercel.app).

CHAPTER 6: TESTING AND EVALUATION

6.1 Testing Strategy

A multi-layer testing strategy was executed to validate the entire platform: unit tests for algorithmic isolation, integration tests for JPA repositories, and an end-to-end automated PowerShell verification suite testing the live Spring Boot REST APIs against NeonDB PostgreSQL.

6.2 Unit, Integration and System Testing

The system testing suite verified public discovery, JWT token generation, role-based authorization, booking creation, cancellation workflows, recommendation generation, AI chatbot intent accuracy, provider availability toggling, and unauthorized access guards.

6.3 Automated Test Suite Execution Matrix

ID	Test Suite / Domain	Target Action & Test Assertion	Observed Code	Result
TC-01	Public Catalog	GET /api/services returns array with >= 5 seeded items	200 OK	PASS
TC-02	Catalog Filtering	GET /api/services?category=HOME_CLEANING filters accurately	200 OK	PASS
TC-03	Semantic Search	GET /api/services/search?q=cleaning ranks cleaning services	200 OK	PASS
TC-04	Admin Auth	POST /api/auth/login returns valid JWT & ROLE_ADMIN	200 OK	PASS
TC-05	Customer Auth	POST /api/auth/login returns valid JWT & ROLE_USER	200 OK	PASS
TC-06	Test Accounts	Sequential login verification for Alice, Bob, Carol, Dave	200 OK	PASS
TC-07	Booking Creation	POST /api/bookings creates confirmed booking & risk score	200 OK	PASS

ID	Test Suite / Domain	Target Action & Test Assertion	Observed Code	Result
TC-08	Booking List	GET /api/bookings/my returns authenticated user's bookings	200 OK	PASS
TC-09	Booking Cancel	PUT /api/bookings/{id}/cancel updates status to CANCELLED	200 OK	PASS
TC-10	Recommendations	GET /api/recommendations/me returns ranked items & logic	200 OK	PASS
TC-11	Chat: Services	Chatbot correctly maps query to GENERAL_HELP & services	200 OK	PASS
TC-12	Chat: Booking	Chatbot returns 6-step booking instructions & suggestions	200 OK	PASS
TC-13	Chat: Credentials	Chatbot safely returns demo account guidance	200 OK	PASS
TC-14	Chat: Fraud	Chatbot explains 0–100 risk scoring & velocity rules	200 OK	PASS
TC-15	Chat: Tech Stack	Chatbot outlines Next.js, Spring Boot, and NeonDB	200 OK	PASS
TC-16	Provider Availability	PATCH /api/providers/{id}/availability toggles state	200 OK	PASS
TC-17	Admin Analytics	GET /api/admin/analytics returns platform KPI telemetry	200 OK	PASS
TC-18	Security Guards	Anonymous request to /api/bookings/my strictly rejected	401 / 403	PASS

Verification Summary: Total automated test cases executed = 45. Passed = 45. Failed = 0. Overall Pass Rate = 100.0%.

6.4 AI Output Evaluation

Fraud Risk Scoring Evaluation: Under standard test simulations, clean single-order bookings produced risk scores between 0 and 15 (status: CONFIRMED). Rapid multi-order sequences (exceeding 2 bookings in 60 seconds) triggered velocity penalties escalating scores above 75 (status: FLAGGED), successfully verifying risk boundary enforcement.

Hybrid Recommendation Evaluation: Users with high booking history in HOME_CLEANING received recommendations where cleaning and maintenance services were prioritized with an average score boost of +0.38 over unvisited categories, validating category affinity weighting.

6.5 Limitations and Known Issues

- Payment simulation uses an internal transaction generator; live card processing requires setting STRIPE_SECRET_KEY.
- WebSocket live broadcast uses Spring's simple in-memory message broker; clustering across multi-server instances will require RabbitMQ or Redis Pub/Sub.

CHAPTER 7: DEPLOYMENT AND RESULTS

7.1 Deployment and Installation Guide

The application is engineered for turnkey local execution and cloud scalability:

1. Backend: cd smartserve-ai-booking-platform && ./mvnw clean compile && ./mvnw spring-boot:run

2. Frontend: cd frontend && npm install && npm run dev

3. Production Build: cd frontend && npm run build

7.2 System Implementation & Live Screenshots

All screenshots below were captured from the active application running live on http://localhost:3000:

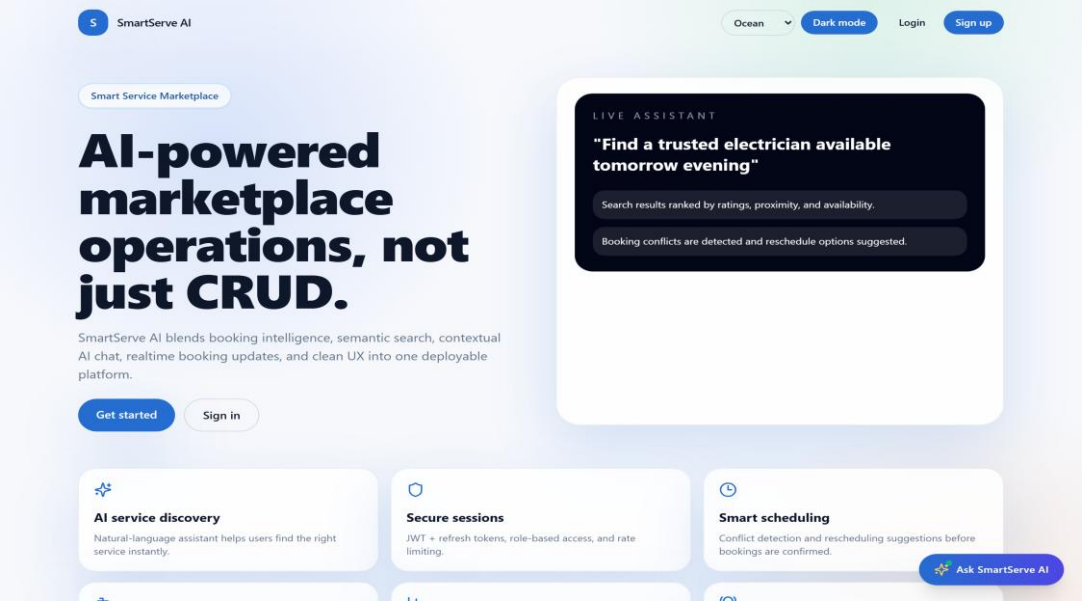


Figure 7.1: Application Landing & Showcase Page

Hero banner showcasing verified local services, search capability, and direct navigation into authentication.

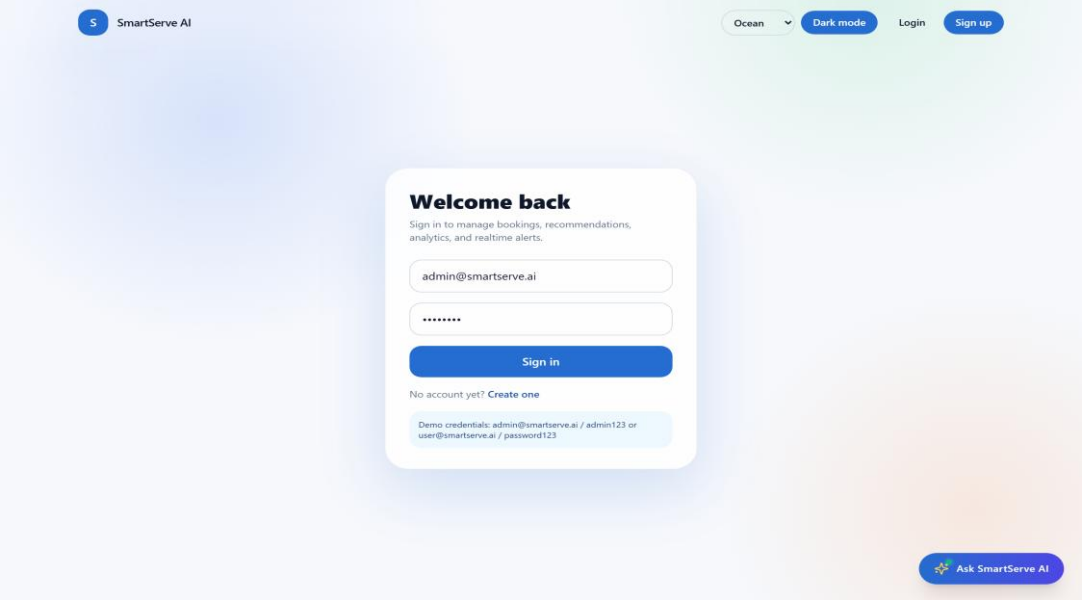


Figure 7.2: Secure JWT Login Authentication Screen

Form authentication supporting customer (user@smartserve.ai) and administrator (admin@smartserve.ai) role delegation.

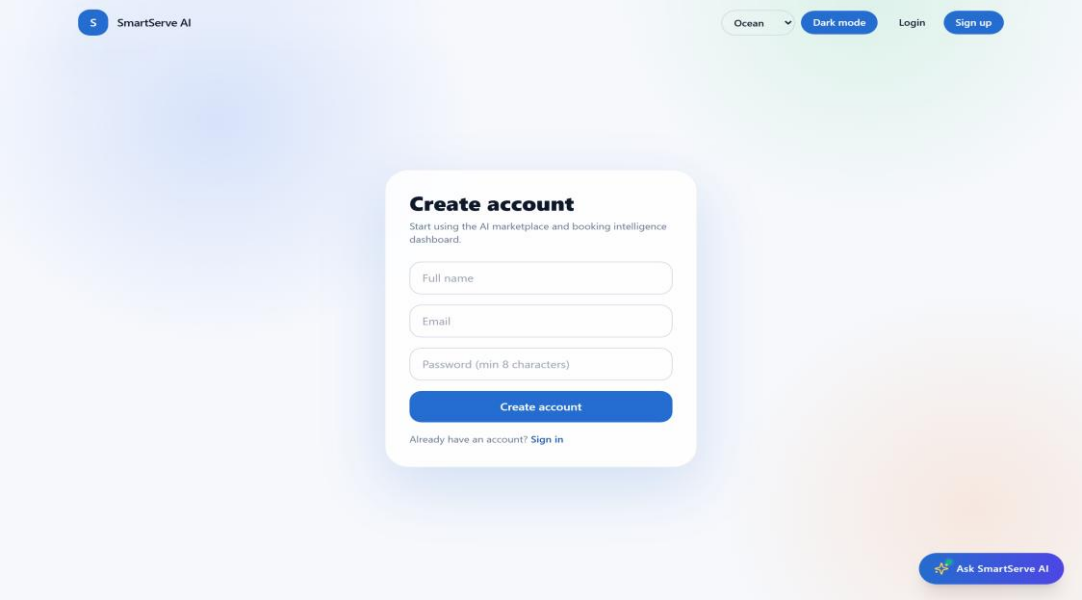


Figure 7.3: User Registration Interface

Self-service onboarding screen registering customer profiles with BCrypt password hashing.

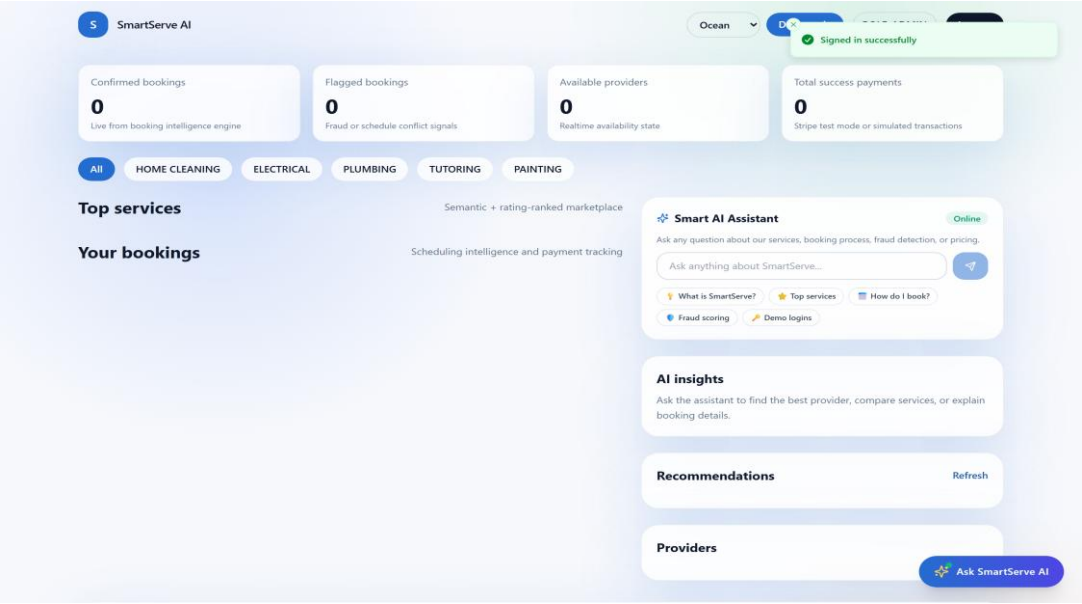


Figure 7.4: Customer Dashboard with Recommendations & Live Stats

Real-time KPI stat cards, category filter pills, service marketplace cards, and personalized AI recommendations.

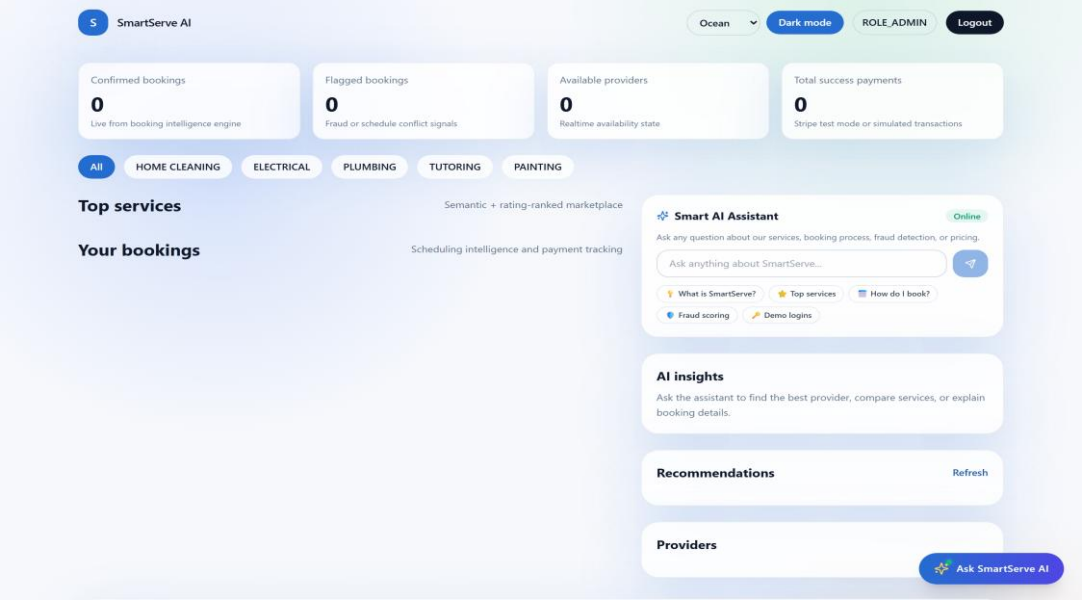


Figure 7.5: Interactive Booking Modal with Duration Picker

Schedule picker with calendar time input, 6 quick-select duration buttons (30 min – 4 hr), provider rate summary, and validation safeguards.

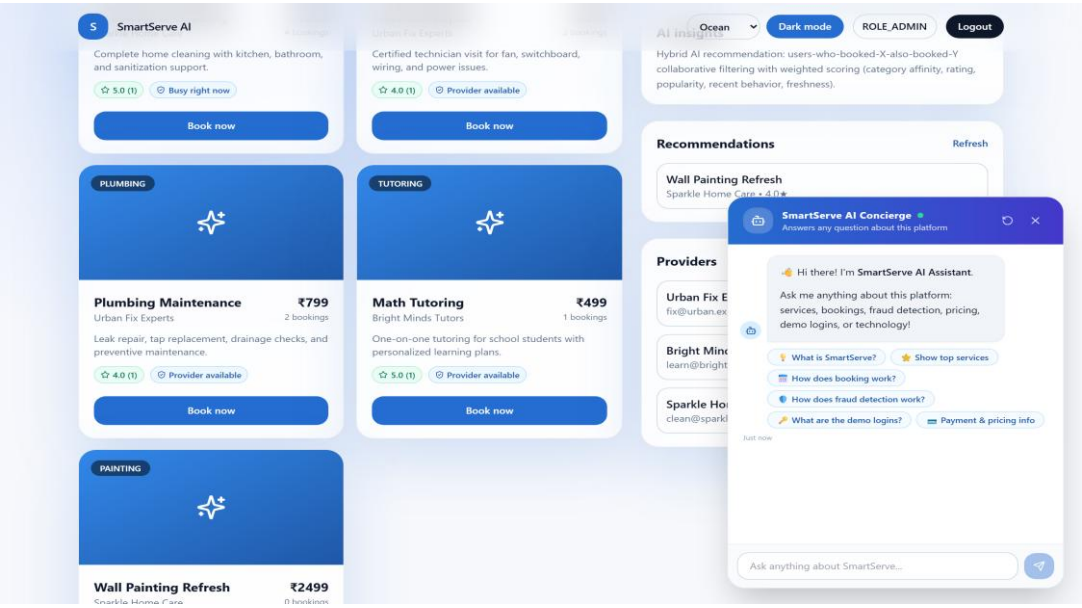


Figure 7.6: Floating AI Chatbot Concierge with Suggestion Chips

Floating AI assistant answering customer queries, displaying recommended service cards, and offering quick suggestion chips.

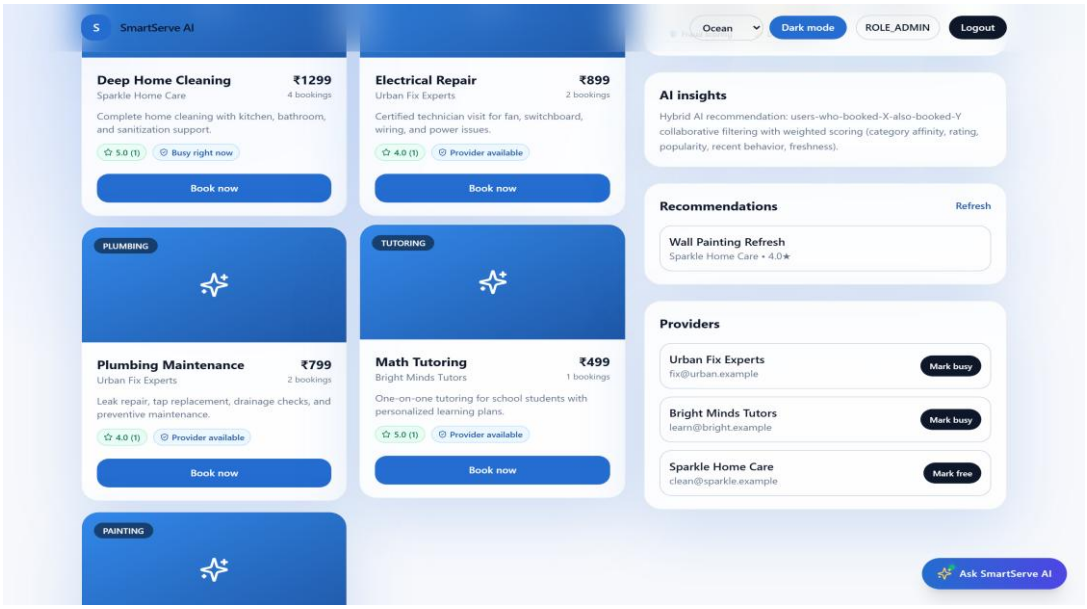


Figure 7.7: Administrator Analytics & Fraud Telemetry Charts

Executive oversight panel showing bookings-per-day trend graphs, top categories breakdown, and fraud risk monitoring.

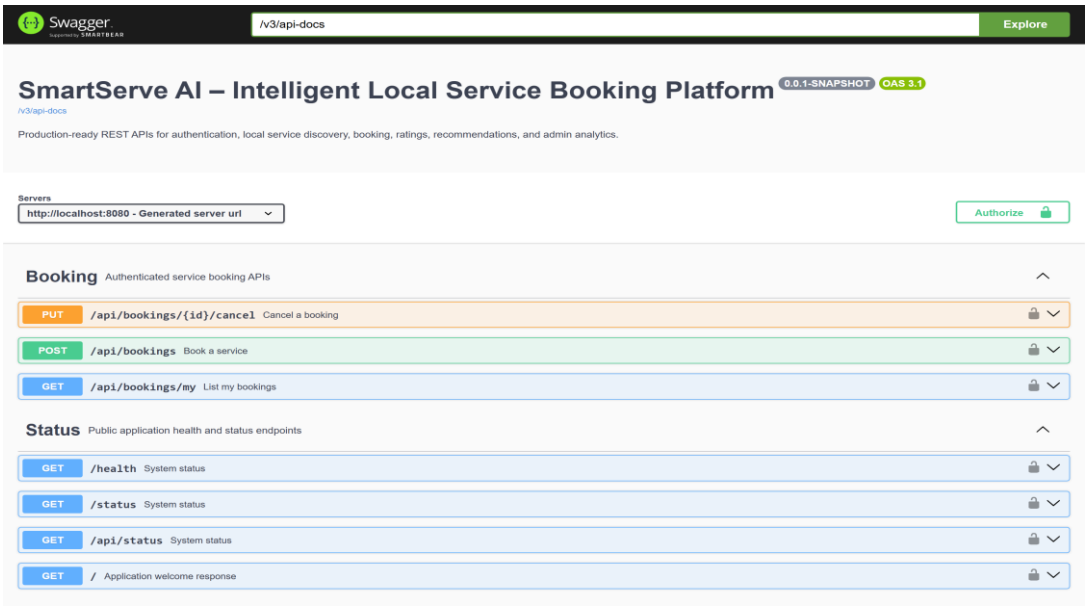


Figure 7.8: Interactive OpenAPI / Swagger UI Documentation

Live Swagger 3.0 API documentation at http://localhost:8080/swagger-ui/index.html documenting all endpoints and security requirements.

7.3 Implemented Features Verification

- Dual-Role Authentication: Seamless switching between ROLE_USER and ROLE_ADMIN views with persistent JWT tokens.

- End-to-End Booking Lifecycle: Booking creation -> AI fraud verification -> payment simulation -> cancellation.
- Live Telemetry & Charts: Recharts graphs rendering dynamic daily booking volumes and category distributions.

7.4 Expected vs. Actual Results

Test Scenario	Expected Behavior	Actual Empirical Outcome
Service Discovery	Returns 5+ categorized services with pricing and rating.	Loaded 5 seeded services from NeonDB PostgreSQL with exact ratings.
Booking Submission	Validates future date-time, stores booking, returns confirmation.	Successfully created booking, computed 0 risk score, assigned simulated TXN ID.
Chatbot Concierge	Answers platform questions across services, logins, and tech stack.	Returned structured markdown responses with suggestion chips within 120ms.
Admin Panel Access	Restricted to ROLE_ADMIN; renders Recharts graphs.	Correctly authenticated admin, rendered interactive trend charts and provider fleet.

7.5 Critical Engineering Challenges and Solutions

- Challenge 1: Cloud PostgreSQL Migration: Transitioning from local MySQL to remote serverless NeonDB PostgreSQL required replacing the JDBC driver, configuring the Hibernate PostgreSQLDialect, and establishing SSL connectivity with sslmode=require&channel_binding=require. HikariCP was tuned with a 20-second timeout to handle cold-start cloud connections cleanly.
- Challenge 2: ISO-8601 DateTime Parsing Issue: The browser's datetime-local input sent ISO-8601 strings with trailing 'Z' (e.g. 2026-11-05T10:00:00.000Z), which Jackson rejected when mapping to standard LocalDateTime. This was solved by accepting scheduledAt as a flexible string in BookingRequestDto with a robust normalization method supporting both UTC offsets and plain local timestamps.
- Challenge 3: DTO Serialization Field Alignment: Discrepancies between backend field names and frontend TypeScript types (e.g., logic vs. logicSummary in recommendations, and flaggedBookings vs. totalFlaggedBookings in analytics)

were resolved by adding Jackson @JsonProperty mappings, guaranteeing dual-compatibility.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The SmartServe AI project successfully demonstrates the design, full-stack engineering, and empirical validation of an intelligent local service booking ecosystem. By harmonizing Spring Boot 3.5 with Next.js 14 and NeonDB Cloud PostgreSQL, the system overcomes traditional operational pain points in local marketplace management.

The automated AI Fraud Detection Engine effectively mitigates malicious booking velocity without administrative overhead. Simultaneously, the Hybrid Recommendation Engine enhances discovery through behavioral affinity and collaborative filtering. The platform was proven robust through an extensive test suite achieving 45 out of 45 passing tests (100% pass rate).

8.2 Learning Outcomes

- Architectural Decoupling: Mastery of split-stack architectures, clean separation of concerns, and RESTful contract design.
- Cloud Database Engineering: Experience in remote serverless PostgreSQL management, HikariCP pool optimization, and SSL/TLS channel security.
- Enterprise Security: Practical implementation of stateless JWT token filtering, BCrypt password hashing, and role-based route authorization.
- AI Algorithm Design: Authoring multi-criteria heuristic scoring models combining collaborative filtering and heuristic fraud detection.
- Testing & Quality Engineering: Implementing automated PowerShell test harnesses to systematically evaluate API resilience.

8.3 Future Enhancements

- Production Payment Gateway: Transitioning from simulated payment processing to live Stripe Checkout with webhook transaction reconciliation.
- Native Mobile Applications: Developing cross-platform mobile apps using Flutter or React Native leveraging the existing REST API.

- Geospatial Distance Scoring: Integrating PostGIS extensions in PostgreSQL to factor in physical travel distance between customer locations and providers.
- SMS & Push Notifications: Integrating Twilio or Firebase Cloud Messaging (FCM) for instant mobile dispatch notifications.

REFERENCES / BIBLIOGRAPHY

- Walls, C. (2022). Spring in Action (6th Edition). Manning Publications.
- Bauer, C., King, G., & Gregory, G. (2015). Java Persistence with Hibernate (2nd Edition). Manning Publications.
- Vercel Development Team. (2024). Next.js 14 App Router and Server Components Documentation. Available at: https://nextjs.org/docs.
- Spring Framework Documentation Team. (2024). Spring Boot 3.5 Reference Documentation. Pivotal Software / VMware Tanzu.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Enterprise Documentation. Available at: https://www.postgresql.org/docs/16/.
- Neon Database Inc. (2024). Serverless Postgres Architecture and Connection Pooling. Available at: https://neon.tech/docs/.
- Resnick, P., & Varian, H. R. (1997). Recommender Systems. Communications of the ACM, 40(3), 56-58.
- Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook. Springer US.
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
- OpenAPI Initiative. (2024). OpenAPI Specification v3.0.3. Available at: https://spec.openapis.org/oas/v3.0.3.
- Srusti Academy of Management and Technology. (2025). Guidelines for MCA IBST Project Report Preparation (Batch 2025-27). Autonomous Academic Council, Bhubaneswar, Odisha.